

Feature engineering and preprocessing

The streaming feature engine shared by every model, the feature sets, inputs, targets, closed-loop protocol and int8 export.

All models get their inputs from one streaming engine (src/utils.py, numba kernel `_push`): it consumes one RR interval per beat and keeps O(1) state, so training, evaluation and the pacemaker compute identical features. It is the reference for the C port. The legacy TCN_MHA is the only exception (see the end).

1. Per-beat streaming engine

- Window: the last $W = 30$ RR intervals (ms), kept in a ring buffer. Cost per beat: $O(W)$ window statistics + $O(p^2)$ RLS update ($p = 8$, about 152 FLOPs) + the RSA bank (about 180 FLOPs) + a biquad (5 multiplies).
- RLS AR(8): predicts the next dRR from the centred RR lags ($RR[n-k] - \text{window mean}$) / 100. Forgetting factor 0.99 (about 100-beat memory), initial covariance 100, covariance-trace cap 10000, forecast clamp ± 300 ms. Stability guard: the update is skipped when the covariance loses positive definiteness, the covariance is kept symmetric and reset if its trace becomes non-finite (without it the RLS diverged after about 3,200 beats on real data).
- Fixed RSA band-pass: RBJ biquad on the raw RR, centre 0.3 cycles/beat, Q 1.0 (about 0.15-0.45 cycles/beat), zero DC gain.
- RSA tracker (round 2): a bank of 8 RBJ band-passes from 0.12 to 0.44 cycles/beat (Q 3.0), band-power EMAs with factor 0.99. The peak band above the in-band log-power trend gives frequency, amplitude, phase and presence. In the closed loop its band powers are frozen on filled beats and a gate sets $rsa_q = rsa_drr = 0$ once 20 or more of the last 30 beats were filled (the RSA phase cannot be recovered there).

2. Engineered features (35, in FEATURE_NAMES order)

feature	definition
rr_mean	mean RR of the 30-beat window
rr_dev	newest RR minus the window mean
rmssd	root mean square of the 29 successive differences
rs	rescaled range: $(\max - \min \text{ of the cumulative sum of } RR - \text{mean}) / \text{std}$
ccm	complex correlation measure: Poincare-triangle area / $(\pi \text{ SD1 SD2 } (W-3))$, clipped to $[0, 1]$
guzik	Guzik asymmetry: $\text{sum of positive dRR} / \text{sum of } dRR $
porta	Porta asymmetry: $\text{count of negative dRR} / \text{count of non-zero dRR}$
nn20	count of $ dRR > 20$ ms in the window
run_len	signed length of the current run of same-sign dRR (+ = decelerating)
rls_pred	RLS AR(8) forecast of the next dRR (ms), clamped to ± 300
rls_err	last RLS innovation: true dRR minus the previous forecast (ms)
bp	RSA band-pass output: RBJ biquad on raw RR, centre 0.30 cycles/beat, Q 1
bp_prev	previous band-pass output (gives the phase with bp)
drr_lag0..7	drr_lag0..7: the 8 most recent dRR (lag0 = newest)
rls_w0..7	rls_w0..7: current RLS AR(8) weights
rsa_f	tracked RSA frequency (cycles/beat): peak of an 8-band resonator bank, parabolic interpolation
rsa_amp	RSA amplitude (ms) from the peak band's mean power
rsa_cos	cos of the current RSA phase (2-sample quadrature of the peak band)
rsa_sin	sin of that phase
rsa_q	RSA presence: peak height above the in-band log-power trend ($\log_2$); 0 when gated
rsa_drr	predicted RSA part of the next dRR: $\text{amp} (\cos(\text{phase} + 2 \pi f) - \cos(\text{phase}))$; 0 when gated

3. How each feature is computed

Notation: when beat n arrives, the window holds its last $W = 30$ RR intervals, RR_0 (oldest) to $RR_{29} = RR[n]$ (newest), in ms. $d_i = RR_{i+1} - RR_i$ are its 29 successive differences (dRR). Every feature of beat n uses only beats up to n (causal) and is the input for predicting $dRR[n+1]$. Window statistics are recomputed from the 30 values at each beat; the RLS, the band-pass and the RSA tracker are recursive: they keep a small state that is updated once per beat. Every formula below is checked when this document is generated: a NumPy re-implementation written from these formulas must reproduce the engine on 4000 beats of validation subject 16273 (relative difference below $1e-6$ for all 35 features).

3.1 Level and variability of the window

rr_mean (ms)

$$rr_mean = (1/30) \sum_{i=0..29} RR_i$$

The local heart period. It is also the centre the RLS regressors are measured from.

rr_dev (ms)

$$rr_dev = RR_{29} - rr_mean$$

How far the newest beat sits from the local level. A positive value (a long beat) tends to be followed by a shortening, so it carries mean reversion.

rmssd (ms)

$$rmssd = \sqrt{(1/29) \sum_{i=0..28} d_i^2}$$

Root mean square of the 29 successive differences: the short-term (beat-to-beat) variability, the classic vagal index. It sets the expected size of the next dRR.

nn20 (count, 0-29)

$$nn20 = \#\{i : |d_i| > 20 \text{ ms}\}$$

How many of the 29 differences exceed 20 ms (strictly). A count, not a percentage.

rs (rescaled range, no unit)

$$C_k = \sum_{i=0..k} (RR_i - rr_mean), k = 0..29; rs = (\max_k C_k - \min_k C_k) / s$$

s is the population standard deviation of the window ($\sqrt{(1/30) \sum (RR_i - rr_mean)^2}$). The cumulative deviation C_k wanders far when the RR stay on one side of the mean for long stretches (a trend or a slow wave) and stays small when they alternate, so rs measures persistence (Hurst-type analysis). rs = 0 if all 30 RR are equal.

3.2 Shape of the beat-to-beat changes

guzik (0-1)

$$guzik = \sum_{d_i > 0} d_i / \sum_i |d_i|$$

Share of the total beat-to-beat change that is lengthening (deceleration). 0.5 means symmetric accelerations and decelerations; 0 if all differences are zero.

porta (0-1)

$$porta = \#\{d_i < 0\} / \#\{d_i \neq 0\}$$

Share of the non-zero differences that are shortenings (accelerations). Zero differences are ignored; 0 if all are zero.

run_len (signed count, -29 to +29)

$$run_len = \text{sign}(d_{28}) \cdot (\text{number of consecutive differences, counted back from } d_{28}, \text{ with that same sign})$$

+3 means the last 3 beats each got longer (decelerating for 3 beats); -2 means the last 2 got shorter. 0 if the newest difference is exactly 0. It tells the model where it is inside a slow wave.

drr_lag0..7 (ms)

$$drr_lag_k = RR_{29-k} - RR_{28-k} = d_{28-k}, k = 0..7$$

The 8 most recent beat-to-beat changes, newest first (drr_lag0 is the dRR that just happened).

ccm (complex correlation measure, 0-1)

$$\text{Poincaré: } P_i = (RR_i, RR_{i+1}), i = 0..28; SD1 = \text{std}_i((RR_i - RR_{i+1})/\sqrt{2}), SD2 = \text{std}_i((RR_i + RR_{i+1})/\sqrt{2})$$

$$A_i = \text{area of the triangle } P_i, P_{i+1}, P_{i+2} = \frac{1}{2} |u_x v_y - u_y v_x|, u = P_{i+1} - P_i, v = P_{i+2} - P_i, i = 0..26$$

$$ccm = \text{clip}(\sum_i A_i / (\pi \cdot SD1 \cdot SD2 \cdot 27), 0, 1)$$

The 29 Poincaré points form 27 triangles of consecutive points. If the RR follow a smooth, predictable pattern the points lie on a line and the triangles are flat; irregular, non-linear dynamics open them up. The sum of their areas is normalised by the area of the SD1 x SD2 Poincaré ellipse (std = population standard deviation over the 29 points). 0 if SD1 or SD2 is 0.

3.3 Adaptive RLS AR(8) forecast

A per-patient autoregressive model whose 8 weights w are re-estimated at every beat by recursive least squares. It runs in three steps per beat:

- **1. Error of the previous forecast** (when beat n arrives): $e = (RR[n] - RR[n-1]) - \text{pred}_{n-1}$, using the unclamped forecast. This is **rls_err** (ms); 0 for the first window.

- **2. Update** with the regressor vector ϕ of the previous forecast: $g = P\phi / (\lambda + \phi^T P \phi)$, $w \leftarrow w + g \cdot e/100$, $P \leftarrow (P - g\phi^T P) / \lambda$, with $\lambda = 0.99$ (about 100 beats of memory), $P_0 = 100 I$ and $w_0 = 0$. Guards: the update is skipped and P reset to P_0 if $\phi^T P \phi < 0$ or its trace is not finite; P is symmetrised each beat and scaled down if its trace exceeds 10000.
- **3. New forecast** on the window that now ends at beat n : $\phi_k = (RR_{29-k} - rr_mean) / 100$, $k = 0..7$ (the last 8 RR, centred on the window mean); $pred = 100 \cdot \sum_k w_k \phi_k$. **rls_pred** = clip(pred, -300, +300) ms, the forecast of dRR[n+1]. **rls_w0..7** are the weights w just used (dimensionless): they describe the patient's current dynamics.

3.4 Fixed RSA band-pass

$$y[n] = b_0 (RR[n] - RR[n-2]) - a_1 y[n-1] - a_2 y[n-2]$$

A second-order RBJ band-pass run on the raw RR series: centre $f_0 = 0.3$ cycles/beat (3.3 beats per breath), $Q = 1.0$, so $\omega_0 = 2\pi f_0$, $\alpha = \sin \omega_0 / 2Q$, $b_0 = \alpha/(1+\alpha) = 0.32228$, $a_1 = -2\cos \omega_0/(1+\alpha) = 0.41886$, $a_2 = (1-\alpha)/(1+\alpha) = 0.35545$. Gain 1 at f_0 and 0 at both ends (constant level and 0.5 cycles/beat), so no detrending is needed; the -3 dB band is about 0.15-0.45 cycles/beat. The input history starts at the first RR, so there is no start-up step. **bp** = $y[n]$ and **bp_prev** = $y[n-1]$ (ms): together they give the amplitude and phase of the respiratory oscillation.

3.5 Patient-adaptive RSA tracker

- **Bank**: 8 band-passes of the same form with $Q = 3.0$ and centres $f_k = 0.120, 0.166, 0.211, 0.257, 0.303, 0.349, 0.394, 0.440$ cycles/beat ($\Delta f = 0.0457$). Each band's power is an exponential average $p_k \leftarrow \beta p_k + (1-\beta) y_k^2$, $\beta = 0.99$ (about 100 beats); in the closed loop it is frozen on filled beats.
- **rsa_q** (log2 units): $L_k = \log_2(p_k + 1e-8)$; a least-squares line $L \sim a + b \cdot f$ is fitted over the 8 bands (the background spectrum); the residual $r_k = L_k - \text{line}(f_k)$ is searched over the 6 interior bands, and $rsa_q = \max_k r_k$: how far the strongest band rises above the trend (1 = twice the power). The peak band is k^* .
- **rsa_f** (cycles/beat): parabolic interpolation of L around k^* : $\text{offset} = \frac{1}{2} (L_{k^*-1} - L_{k^*+1}) / (L_{k^*-1} - 2L_{k^*} + L_{k^*+1})$ when the denominator is negative (a true peak), else 0, clipped to ± 0.5 ; $rsa_f = f_{k^*} + \text{offset} \cdot \Delta f$.
- **rsa_amp** (ms): $\sqrt{2 p_{k^*}}$, the amplitude of a sinusoid whose mean power is p_{k^*} .
- **rsa_cos**, **rsa_sin**: the phase ϕ of the peak band's oscillation from its last two outputs, treating them as samples of $A \cos(\phi)$ and $A \cos(\phi - \omega)$ with $\omega = 2\pi \cdot \text{rsa_f}$: $A \cos \phi = y[n]$, $A \sin \phi = (y[n-1] - y[n] \cos \omega) / \sin \omega$; then normalised to $\cos \phi$ and $\sin \phi$ (1 and 0 if the band is silent).
- **rsa_drr** (ms): the RSA part of the next change, $rsa_amp \cdot (\cos(\phi + \omega) - \cos \phi)$: how much the respiratory sinusoid alone would move the next RR.
- **Gate** (closed loop only): when 20 or more of the last 30 beats were filled by the model, $rsa_q = rsa_drr = 0$, because the phase can no longer be recovered. Training and the open loop never fill, so the gate never acts there.

3.6 Training-split statistics

Mean and standard deviation of each feature over the 20.8M training windows, as fitted by the normalisation (fit_norm) and stored in runs/gbdt_all and runs/mlp_rsa. Every model input is (value - mean) / std.

feature	unit	mean	std
rr_mean	ms	544.2	166.8
rr_dev	ms	0.0007234	45.98
rmssd	ms	29.73	34.12
rs	none	9.317	2.512
ccm	none	0.2404	0.1144
guzik	none	0.4983	0.05821
porta	none	0.5081	0.0688
nn20	count	8.424	7.485
run_len	signed count	-0.04145	1.67
rls_pred	ms	0.4763	29.4
rls_err	ms	-0.4234	39.24
bp	ms	-6.263e-07	20.64
bp_prev	ms	2.497e-06	20.64
drr_lag0	ms	5.059e-06	45.25
drr_lag1	ms	2.245e-05	45.25
drr_lag2	ms	3.724e-05	45.25
drr_lag3	ms	5.102e-05	45.25
drr_lag4	ms	9.684e-06	45.25
drr_lag5	ms	-3.324e-06	45.25

feature	unit	mean	std
drr_lag6	ms	6.644e-05	45.25
drr_lag7	ms	3.517e-06	45.25
rls_w0	none	-0.4659	0.2997
rls_w1	none	0.08505	0.243
rls_w2	none	0.1136	0.1632
rls_w3	none	0.084	0.161
rls_w4	none	0.002521	0.1487
rls_w5	none	-0.03612	0.1191
rls_w6	none	-0.01794	0.1076
rls_w7	none	-0.003419	0.09035
rsa_f	cycles/beat	0.2727	0.05865
rsa_amp	ms	16.69	17.77
rsa_cos	none	0.00364	0.7094
rsa_sin	none	0.002818	0.7047
rsa_q	log2	0.757	0.524
rsa_drr	ms	-0.1011	24.63

3.7 Worked example: validation subject 16273, beat 2000

The window ($RR_0 \dots RR_{29}$, ms) and the 29 differences d_i :

i	0	1	2	3	4	5	6	7	8	9
RR 0-9	844	843	836	828	805	820	789	704	734	742
RR 10-19	750	727	797	804	836	805	844	843	797	813
RR 20-29	781	828	836	805	836	843	829	757	829	843
d 0-9	-1	-7	-8	-23	+15	-31	-85	+30	+8	+8
d 10-19	-23	+70	+7	+32	-31	+39	-1	-46	+16	-32
d 20-28	+47	+8	-31	+31	+7	-14	-72	+72	+14	

Hand check of the simplest ones: $rr_mean = 24148 / 30 = 804.93$; $rr_dev = 843 - 804.93 = 38.07$; $rmssd = \sqrt{(37467 / 29)} = 35.94$; $nn20 = 16$; $drr_lag0 = 843 - 829 = +14$; $run_len = +2$.

feature	value	feature	value
rr_mean	804.9	drr_lag5	31
rr_dev	38.07	drr_lag6	-31
rmssd	35.94	drr_lag7	8
rs	9.942	rls_w0	-0.5168
ccm	0.2278	rls_w1	0.1975
guzik	0.4994	rls_w2	0.133
porta	0.4828	rls_w3	0.2139
nn20	16	rls_w4	-0.3761
run_len	2	rls_w5	-0.1739
rls_pred	-32.82	rls_w6	0.1093
rls_err	33.99	rls_w7	0.09765
bp	31.59	rsa_f	0.2611
bp_prev	15.92	rsa_amp	16.19
drr_lag0	14	rsa_cos	0.9861
drr_lag1	72	rsa_sin	0.1664
drr_lag2	-72	rsa_q	1.059
drr_lag3	-14	rsa_drr	-19.76
drr_lag4	7		

Here the RLS forecasts $dRR[n+1] = -32.8$ ms; the true next change was +9 ms. The tracker sees an RSA peak at 0.261 cycles/beat (3.8 beats per breath), 1.06 log2 above the background, amplitude 16.2 ms, and expects the oscillation alone to move the next RR by -19.8 ms.

4. Feature sets

set	n	features	used by
seq	9	rmssd, rs, ccm, guzik, rls_pred, rls_err, bp, bp_prev, run_len	micro_tcn, micro_tcn_f8, micro_tcn_dad, tcn_lite, tcn_lite_ft, tcn_lite_dad, gru
tab	19	rr_dev, rr_mean, drr_lag0, drr_lag1, drr_lag2, drr_lag3, drr_lag4, drr_lag5, drr_lag6, drr_lag7, rmssd, rs, ccm, guzik, rls_pred, rls_err, bp, bp_prev, run_len	mlp, gbd
ar	9	rr_dev, drr_lag0, drr_lag1, drr_lag2, drr_lag3, drr_lag4, drr_lag5, drr_lag6, drr_lag7	linear
legacy_chenp	6	rmssd, rs, ccm, guzik, porta, nn20	(not used by a trained run)
tab_nobp	17	rr_dev, rr_mean, drr_lag0, drr_lag1, drr_lag2, drr_lag3, drr_lag4, drr_lag5, drr_lag6, drr_lag7, rmssd, rs, ccm, guzik, rls_pred, rls_err, run_len	mlp_nobp
tab_rsa	23	rr_dev, rr_mean, drr_lag0, drr_lag1, drr_lag2, drr_lag3, drr_lag4, drr_lag5, drr_lag6, drr_lag7, rmssd, rs, ccm, guzik, rls_pred, rls_err, run_len, rsa_f, rsa_amp, rsa_cos, rsa_sin, rsa_q, rsa_drr	mlp_rsa
seq_rsa	13	rmssd, rs, ccm, guzik, rls_pred, rls_err, bp, bp_prev, run_len, rsa_f, rsa_amp, rsa_q, rsa_drr	tcn_lite_rsa, tcn_lite_rsa_ws, micro_tcn_rsa, micro_tcn_rsa_ws
all	35	all features (FEATURE_NAMES)	gbd_all (29 features: trained before the 6 RSA features were appended)

5. Model inputs, normalisation and target

- **Sequence input** (TCN/GRU): the last 30 RR intervals as two channels, z-RR and z-dRR (first step dRR = 0), z-scored with mean/std fitted on the training subjects only (zero variance -> std 1).
- **Tabular input**: the engineered features of the newest beat (see 01_feature_engineering.pdf), z-scored per feature with training statistics.
- **Target**: $dRR[n+1] = RR[n+1] - RR[n]$. Residual models predict the z-scored correction over the RLS forecast ($dRR - rls_pred$); the output is de-normalised and added back to rls_pred .
- The first 30 windows of every series are dropped from training and scoring (cold RLS state).

6. Closed-loop preprocessing (the pacemaker use, notebooks/correlation_study_v3.ipynb)

- Dropped beats: a random share (10/30/50/80%) of beats is removed, uniformly from beat 0 (as `utils2.random_extraction` with `start_idx = 0`); the same mask is used for every model.
- First 30 beats (no full window yet): filled by the notebook's Burg-AR predictor (AR(15) coefficients from `data/ar_model_parameters.json`, exact finite-history MMSE weights, anchored to the age-matched RR reference $505 \times \text{age}^{0.122}$ ms); these fills are not scored.
- From beat 30: each dropped beat is filled by the model; the fill enters the window, the features and the RLS state of later beats. Only model-filled beats are scored.
- RSA fidelity: Welch PSD of the filled vs the original series in 0.15-0.40 cycles/beat; a peak counts as RSA when it is at least 2x above the in-band trend; invented = new peak (and grown $\geq 1.4x$), displaced = moved > 0.03 cycles/beat, lost = disappeared.

7. Closed-loop training data (DAD, round 3)

For the `_dad` runs, every training series was cut into 5000-beat segments with a random 10-80% of beats dropped and filled in closed loop by the parent model itself; the engine was run on the filled series and the target was the TRUE next beat ($rr_true[n+1] - rr_filled[n]$). Batches mixed 50% of these windows with 50% ordinary ones. It did not improve the deployed `int8` model significantly.

8. int8 export

Full-integer post-training quantisation with TFLite (int8 weights, activations, input and output, the CMSIS-NN / ST Edge AI format). Calibration is deterministic best-of-K: 8 random draws of 2k or 5k training windows are tried and the export with the smallest int8-vs-float gap on 20 held-out training subjects (ordinary and closed-loop windows) is kept. Validation is never used for calibration.

9. Legacy TCN_MHA features (not from the engine)

The legacy model recomputes its inputs per window with `legacy/utls2.py`: 12 static features (`rmssd`, `rs`, `ccm`, `ccm_n5`, `guzik`, `nn20`, `porta`, `ar_1`, `ar_2`, `ar_3`, `ar_4`, `ar_5`) z-scored with the statistics stored in `data/hrv_dataset.h5`, plus the 30 RR values z-scored with global statistics (one channel). The Burg AR(5) coefficients need an $O(\text{order} \times W)$ fit per beat.

Abbreviations and acronyms used in this document

term	meaning
AI	artificial intelligence
AR	autoregressive: a value predicted as a weighted sum of past values; AR(p) uses p past values
ar	feature-set name: autoregressive inputs (rr_dev and the 8 most recent dRR) of the linear model
bp	feature: RSA band-pass output: RBJ biquad on raw RR, centre 0.30 cycles/beat, Q 1
bp_prev	feature: previous band-pass output (gives the phase with bp)
Burg-AR	AR model whose coefficients were fitted with Burg's method (used for the first 30 beats)
ccm	complex correlation measure: non-linearity index of the Poincare plot of consecutive RR intervals
CMSIS-NN	Cortex Microcontroller Software Interface Standard - Neural Network kernels: Arm's optimised neural-network library for Cortex-M microcontrollers
DAD	data as demonstrator: training on inputs that contain the model's own closed-loop fills, with the true target (a remedy for exposure bias)
_dad	run-name suffix: fine-tuned with DAD (closed-loop-filled) training data
DC	direct current, i.e. the constant (zero-frequency) component of a signal
dRR	beat-to-beat change of the RR interval: $dRR[n+1] = RR[n+1] - RR[n]$ (ms); the quantity predicted
drr_lag0..7	feature: drr_lag0..7: the 8 most recent dRR (lag0 = newest)
EMA	exponential moving average
_f8	run-name suffix: 8 convolution filters instead of 16
FLOPs	floating-point operations (arithmetic cost)
_ft	run-name suffix: fine-tuned (training continued from the parent's weights)
gbdt	gradient-boosted decision trees (run name)
GRU	gated recurrent unit (a recurrent neural-network cell)
gru	gated recurrent unit (run name)
guzik	feature: Guzik asymmetry: sum of positive dRR / sum of dRR
HRV	heart rate variability
int8	8-bit integer: weights and activations quantised to 8-bit integers, as run on the microcontroller; _int8 = the int8 version of a run
log2	base-2 logarithm
MHA	multi-head attention (a transformer layer, not supported by the MCU toolchain)
mlp	multilayer perceptron (run name)
MMSE	minimum mean square error
ms	milliseconds
nn20	number of successive RR differences larger than 20 ms in the window
nobp	run-name part: no band-pass (the fixed RSA band-pass features bp and bp_prev removed)
O(1)	constant cost per beat (big-O notation); O(W) = proportional to the window length W
p	p-value of the paired per-subject Wilcoxon test: probability of a difference at least this large if the two models were equally accurate ($p < 0.05$ = significant); in AR(p), the number of past values
porta	feature: Porta asymmetry: count of negative dRR / count of non-zero dRR
PSD	power spectral density (signal power per frequency)
Q	quality factor of a band-pass filter: centre frequency / bandwidth
RBJ	Robert Bristow-Johnson's standard biquad (second-order) filter formulas
RLS	recursive least squares: an adaptive filter that updates its coefficients at every beat
rls_err	feature: last RLS innovation: true dRR minus the previous forecast (ms)
rls_pred	feature: RLS AR(8) forecast of the next dRR (ms), clamped to +/-300
rls_w0..7	feature: rls_w0..7: current RLS AR(8) weights
rmssd	root mean square of successive differences of the RR intervals in the window
RR	RR interval: time between two consecutive heartbeats (R waves of the ECG), in ms
rr_dev	feature: newest RR minus the window mean
rr_mean	feature: mean RR of the 30-beat window
rs	rescaled range: (max - min of the cumulative deviations from the mean) / standard deviation
RSA	respiratory sinus arrhythmia: the oscillation of the heart rate driven by breathing
_rsa	run-name suffix: uses the RSA-tracker features
rsa_amp	feature: RSA amplitude (ms) from the peak band's mean power
rsa_cos	feature: cos of the current RSA phase (2-sample quadrature of the peak band)
rsa_drr	feature: predicted RSA part of the next dRR: $\text{amp}(\cos(\text{phase} + 2\pi f) - \cos(\text{phase}))$; 0 when gated

term	meaning
rsa_f	feature: tracked RSA frequency (cycles/beat): peak of an 8-band resonator bank, parabolic interpolation
rsa_q	feature: RSA presence: peak height above the in-band log-power trend (log2); 0 when gated
rsa_sin	feature: sin of that phase
run_len	feature: signed length of the current run of same-sign dRR (+ = decelerating)
SD1	Poincare-plot standard deviation across the identity line (short-term variability)
SD2	Poincare-plot standard deviation along the identity line (long-term variability)
seq	feature-set / model-kind name: sequence model (takes the raw RR window plus tabular features)
ST	STMicroelectronics (ST Edge AI: its toolchain for running neural networks on STM32 microcontrollers)
tab	feature-set / model-kind name: tabular model (engineered features only)
TCN	temporal convolutional network (causal, dilated 1-D convolutions)
tcn	temporal convolutional network (run-name part)
TFLite	TensorFlow Lite: the runtime and file format for models on embedded devices
W	window length: the 30 most recent RR intervals
_ws	run-name suffix: warm start (initialised from the parent's weights, new inputs set to zero)
z	z-score: (value - mean) / standard deviation, with statistics from the training subjects